

Reversión de Transformaciones en Imagen BMP mediante Rotaciones y XOR + Enmascaramiento y Verificación

Desafío I - Informática II, Semestre 2025-1

Daniel Rúa, Tomás Mesa

Ingeniería Electrónica
Universidad de Antioquia

Resumen—Este informe detalla el desarrollo de un proyecto para reconstruir una imagen BMP original a partir de una versión distorsionada. Se emplean únicamente operaciones XOR y rotaciones circulares a la derecha sobre bloques de 8 bits, seguidas de enmascaramiento y verificación contra datos de referencia. Se describe el análisis del problema, las tareas definidas, los algoritmos implementados, los obstáculos afrontados y la evolución de la solución.

Index Terms—Imagen BMP, Rotación a la derecha, XOR, Enmascaramiento, Verificación, Ingeniería inversa.

I. ANÁLISIS DEL PROBLEMA Y CONSIDERACIONES

La imagen BMP distorsionada fue procesada originalmente con varias transformaciones a nivel de bits (desplazamientos, rotaciones y XOR) y luego enmascarada. Conforme al diagrama de proceso (Figura 1), decidimos:

- Excluir desplazamientos lógicos (pierden información).
- Excluir rotaciones a la izquierda (equivalentes a rotaciones a la derecha en sentido complementario).
- Conservar rotaciones circulares a la derecha (variables de 1 a 7 bits) y la operación XOR.
- Aplicar enmascaramiento aditivo con una máscara fija y verificar contra archivos de referencia.

Esta alternativa asegura revertir sin pérdida las transformaciones originales.

II. ESQUEMA DE TAREAS DEFINIDAS

Según el flujo de trabajo (Figura 1), establecimos las siguientes tareas:

1. **Carga de datos:** lectura de la imagen distorsionada, la imagen máscara y archivos .txt de referencia.
2. **Transformación:** aplicar XOR (fase $j=0$) o rotación circular a la derecha (fase $j=1\dots 7$).
3. **Enmascaramiento:** sumar el resultado con la máscara respetando un offset (semilla).
4. **Verificación:** comparar cada byte enmascarado con el valor esperado en el archivo .txt.
5. **Iteración:** si verifica, fijar esa transformación, exportar bloque y avanzar al siguiente archivo de referencia; sino, probar la siguiente rotación.

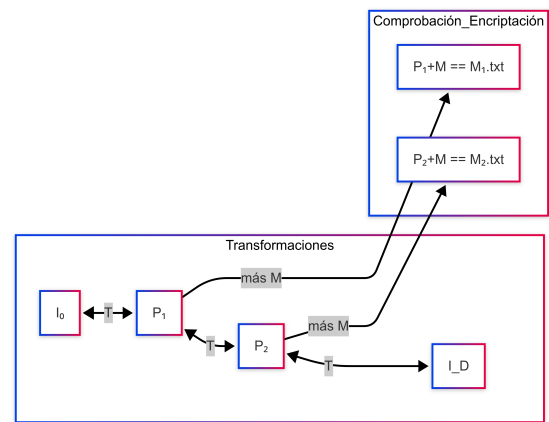


Figura 1. Flujo de transformaciones, enmascaramiento y verificación

III. ALGORITMOS IMPLEMENTADOS

III-A. Transformación (XOR y Rotación Derecha)

`applyTransformation` recibe un bloque de bytes y un índice j :

- Si $j==0$: $\text{out}[i] = \text{in1}[i] \wedge \text{in2}[i]$ (XOR con imagen secundaria).
- Si $1 \leq j \leq 7$: $\text{out}[i] = (\text{in}[i] \gg j) (\text{in}[i] \ll (8-j))$.

III-B. Enmascaramiento

`applyMask` recorre cada byte transformado y realiza:

$$\text{masked}[k + \text{seed}] = \text{transformed}[k + \text{seed}] + \text{mask}[k]$$

con `seed` obtenido del .txt de referencia.

III-C. Verificación

`verifyMaskedData` comprueba byte a byte:

$$\text{masked}[k + \text{seed}] \stackrel{?}{=} \text{ref}[k] \quad \text{mód } 256$$

Si todos coinciden, retorna verdadero.

IV. PROBLEMAS DE DESARROLLO

- Alineación de offsets entre imagen y máscara: se resolvió ajustando el índice con la semilla proveniente de los archivos .txt.
- Gestión de memoria dinámica sin STL: cuidado en liberaciones para evitar fugas.
- Determinar el orden correcto de transformaciones: se utilizó un contador de verificaciones y reinicio de j para iterar rotaciones.
- Sincronización de lectura de múltiples archivos .txt: implementar recarga y liberación de datos tras cada verificación exitosa.
- Muchas veces el programa crasheaba porque, al recargar los datos de verificación, asignábamos memoria sin liberar la anterior, provocando sobreasignación y corrupción de memoria.
- El cálculo incorrecto del tamaño de los búferes (no considerar siempre $\text{extttwidth} \cdot \text{height} \cdot 3$) causó accesos fuera de rango y fallos intermitentes.

V. EVOLUCIÓN DE LA SOLUCIÓN Y CONSIDERACIONES

- Inicialmente se intentaron desplazamientos, pero se perdió información crítica; se descartaron.
- Se mantuvo redundancia al incluir rotaciones a la izquierda, luego se simplificó a solo rotaciones a la derecha.
- Se definió un flujo iterativo que reinicia j tras cada éxito para procesar en cascada múltiples etapas.
- Futuras implementaciones podrían incorporar paralelismo por bloques de píxeles para acelerar la verificación.
- Considerar validar integridad mediante checksum adicional antes y después de cada etapa.
- Una nueva sección de extbfenciptación de imágenes: aplicar XOR y rotaciones de manera controlada para generar distorsiones “a la carta”, y luego verificar distintos métodos de desenciptación.
- Incorporar múltiples casos de prueba con distintas máscaras y secuencias de transformaciones para robustecer el sistema y automatizar la detección de la configuración de cifrado utilizada.

VI. CONCLUSIONES

En este proyecto hemos demostrado que, limitándose a operaciones reversibles (XOR y rotaciones circulares a la derecha), es posible reconstruir íntegramente una imagen BMP distorsionada sin pérdida de información. Los desplazamientos lógicos fueron descartados porque descartan bits (los que salen por un extremo no se recuperan), lo que impide su inversión fiable. Del mismo modo, rotaciones a la izquierda resultan redundantes—una rotación izquierda de k bits equivale a una rotación derecha de $8 - k$ bits—por lo que con rotaciones a la derecha cubrimos todas las posibles reorientaciones de bits.

Las rotaciones circulares a la derecha preservan la integridad de los 8 bits de cada byte, permitiendo revertir exactamente la operación. La XOR es su propia inversa, por lo que también puede deshacerse aplicándola nuevamente con la misma clave (imagen de referencia). El enmascaramiento mediante suma se utilizó únicamente como medio de verificación, no como transformación irreversible.

VII. CÓDIGO FUENTE

Se incluye el código completo en C++/Qt utilizado para cargar, transformar, enmascarar y verificar los datos. Debido a su extensión, se remite al repositorio público con enlace en la documentación adicional.